

EDGE INTELLIGENCE LABORATORY

ASSIGNMENT 8

SUBMITTED BY

SHIBU P

25MML0042

Table of Contents

S.No	Description	Page No
1	Live Video Checking from laptop camera	3
2	Capturing live video from the camera	4-5
3	Loading sample video from Local device	6-7
4	Analyzing the sample offline video's Grayscale, RGB, Negative	8-10
5	Loading Offline sample video and printing total frames of a video,fps,width and height of the frame.	11
6	Loading video and converting into Grayscale video	12
7	Creating Rectangle and circle using opencv	13-14
8	Converting live video to find edge using canny operator	15-19
9	Converting Live video to Negative video:	20-21
10	Detecting the motion if motion found edge will be shown otherwise black screen will be shown.	22-24
11	Number Plate Detection System	25-36

On Device Image Recognition

[Code Link](#)

1.Importing Libraries and Checking live video from laptop camera

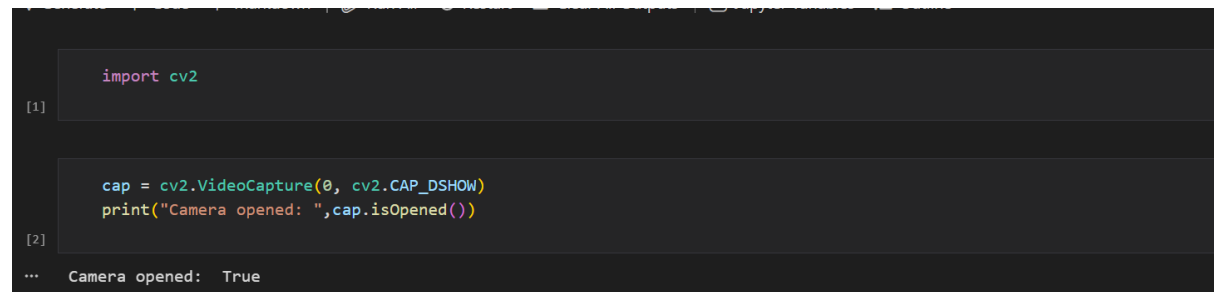
```
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
```

cv2.VideoCapture() is a method

0 is used here for default inbuilt camera

cv2.CAP_DSHOW is used for direct show backend for windows

cap.isOpened() is used to check whether camera works. if works then it returns true otherwise it returns false.



```
[1] import cv2

[2] cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    print("Camera opened: ",cap.isOpened())

... Camera opened: True
```

2.Capturing video from Laptop camera

A while True loop is then used to continuously read frames from the camera. Inside the loop, `ret, frame = cap.read()` captures a single frame, here **ret** is a Boolean value indicating whether the frame was successfully captured, and **frame** contains the actual image data of the captured frame. `cv2.imshow("laptop camera", frame)` displays the video frame in a window titled **“laptop camera.”** The function `cv2.waitKey(1)` waits for 1 millisecond for a keyboard input, and the condition `& 0xFF == ord('q')` checks whether the user pressed the 'q' key. If 'q' is pressed, the loop breaks and the video capture stops. Thus, the program continuously reads and displays frames from the webcam until the user presses the 'q' key to exit.

```
import cv2
#Open default camera(0 = laptop webcam)
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read() #read frame from camera

    if not ret:
        print("Failed to grab frame")
        break

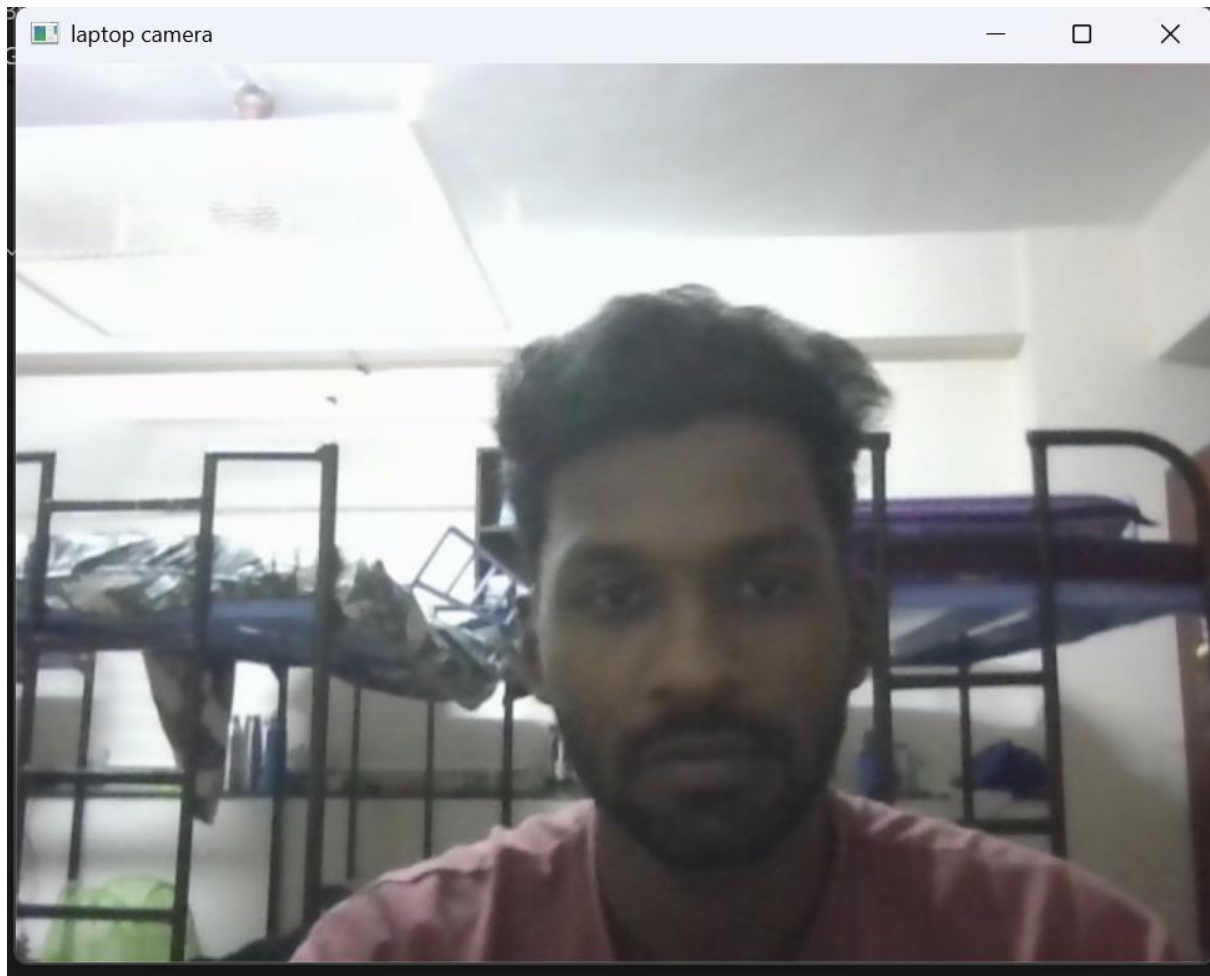
    cv2.imshow("laptop camera",frame) # show the frame

    #press 'q' to exit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

19.4s

Run by Line (F10)

Output:



3.Loading sample video from local device:

Instead of **VideoCapture** parameter as 0 we pass the path of the offline local video.

```
import cv2

cap = cv2.VideoCapture("sample_video.mp4")

while cap.isOpened():
    ret, frame = cap.read()

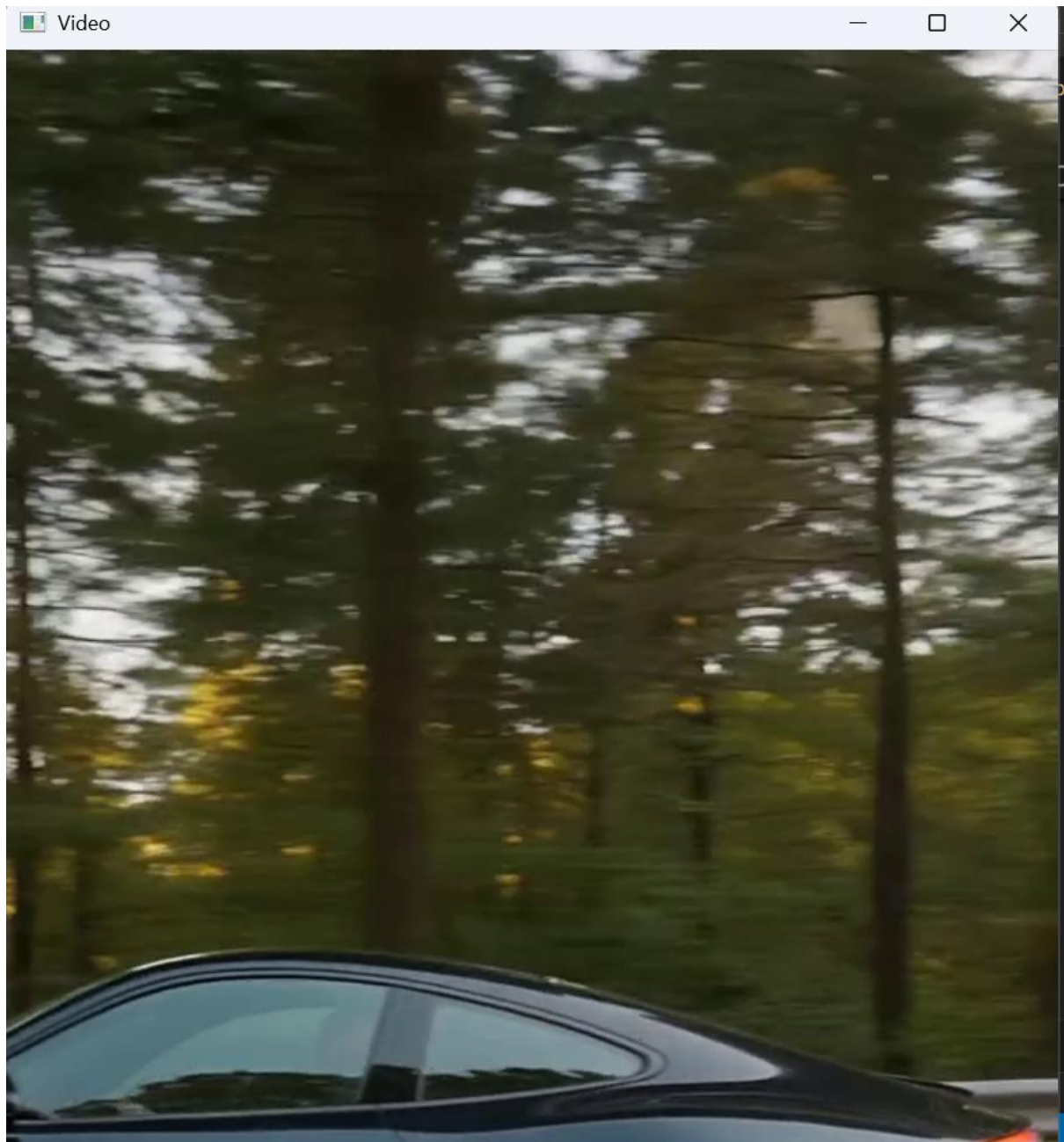
    if not ret:
        break

    cv2.imshow("Video",frame)

    if cv2.waitKey(25) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Output:



4. Analyzing the sample offline video's Grayscale, RGB, Negative

`cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)` converts the frame into a grayscale image, `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` converts the OpenCV BGR format to RGB for proper display in Matplotlib. `negative = 255 - frame`, convert image to negative. `h, w, c = frame.shape` extract the height, width and channel of the image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load video
cap = cv2.VideoCapture("sample_video.mp4") # change to your video path

ret, frame = cap.read()

if not ret:
    print("Error reading video")
    cap.release()
    exit()

# Convert formats
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
negative = 255 - frame

# Display images in output cell
plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.title("Original")
plt.imshow(rgb)
```



```

plt.subplot(2,2,2)
plt.title("Gray")
plt.imshow(gray, cmap="gray")
plt.axis("off")

plt.subplot(2,2,3)
plt.title("RGB")
plt.imshow(rgb)
plt.axis("off")

plt.subplot(2,2,4)
plt.title("Negative")
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.show()

# Print pixel values of first frame
h, w, c = frame.shape

print("Frame Shape:", frame.shape)
print("\nPixel values of first frame:\n")

for i in range(h):
    for j in range(w):
        print(f"Pixel ({i},{j}) BGR:", frame[i,j])

```

✓ 25.2s

Output:



RGB



Negative



Showing pixel values:

```
Pixel (1072,28) BGR: [18 20 13]
Pixel (1072,29) BGR: [19 21 14]
Pixel (1072,30) BGR: [19 21 14]
Pixel (1072,31) BGR: [18 20 13]
Pixel (1072,32) BGR: [19 18 12]
Pixel (1072,33) BGR: [19 18 12]
Pixel (1072,34) BGR: [19 18 12]
Pixel (1072,35) BGR: [19 18 12]
Pixel (1072,36) BGR: [20 19 13]
Pixel (1072,37) BGR: [20 19 13]
Pixel (1072,38) BGR: [20 19 13]
Pixel (1072,39) BGR: [20 19 13]
Pixel (1072,40) BGR: [19 18 12]
Pixel (1072,41) BGR: [19 18 12]
Pixel (1072,42) BGR: [19 18 12]
Pixel (1072,43) BGR: [19 18 12]
Pixel (1072,44) BGR: [20 19 13]
Pixel (1072,45) BGR: [20 19 13]
```

5.Loading Offline sample video and printing total frames of a video,fps,width and height of the frame.

`total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))` obtains the total number of frames present in the video

`fps = cap.get(cv2.CAP_PROP_FPS)`, retrieves the frames per second (FPS) of the video, which indicates how many frames are displayed every second during playback.

The third line, `width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)`, returns the width of each video frame in pixels, while the fourth line

`height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)`, returns the height of each frame in pixels.

```
#Display Video Properties
import cv2

cap = cv2.VideoCapture("sample_video.mp4")

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
fps = cap.get(cv2.CAP_PROP_FPS)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

print("Total frames:",total_frames)
print("fps:",fps)
print("width",width)
print("height:",height)

cap.release()
✓ 0.0s
```

```
Total frames: 241
fps: 24.0
width 612.0
height: 1080.0
```

6.Loading video and converting into Grayscale video:

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

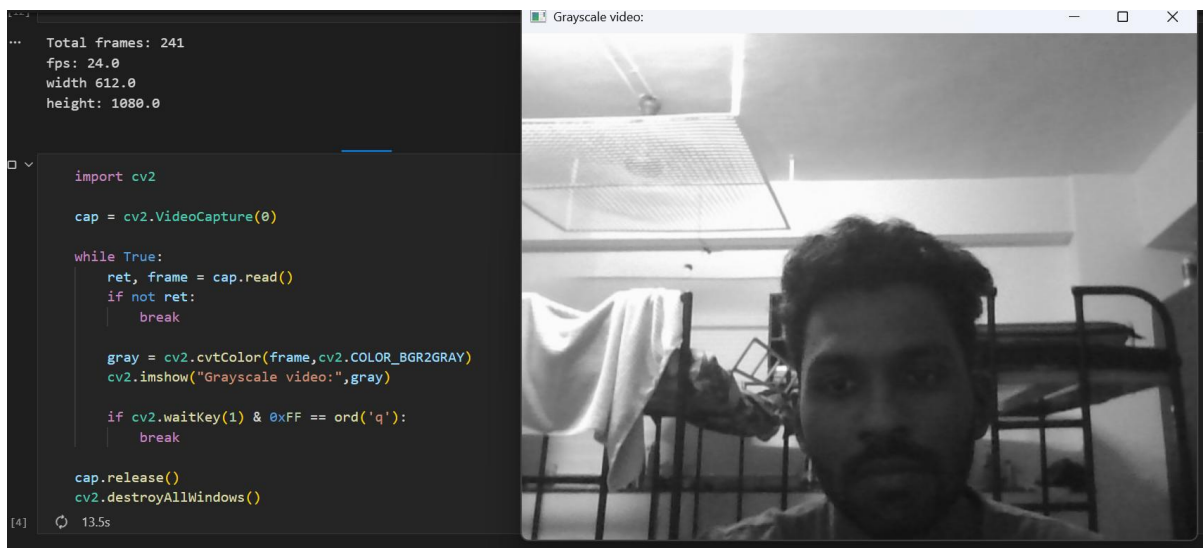
    gray = cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
    cv2.imshow("Grayscale video:",gray)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

✓ 21.7s

Output:



7.Creating Rectangle and circle using opencv

`cv2.rectangle(frame, (100,100), (300,300), (0,255,0), 3)`, draws a rectangle on the image stored in `frame`. The point `(100,100)` represents the **top-left corner** of the rectangle and `(300,300)` represents the **bottom-right corner**. `3` specifies the **thickness of the rectangle border in pixels**. The `cv2.circle(frame, (200,200), 50, (255,0,0), 3)`, draws a **circle** on the frame where `(200,200)` is the **center of the circle**, `50` is the **radius**, `(255,0,0)` represents **blue color in BGR format**, and `3` indicates the **thickness of the circle's outline**. The `cv2.putText(frame, "OpenCV practice", (50,50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)` to display text on the image. Here "OpenCV practice" is the text string, `(50,50)` is the starting position of the text on the image, `cv2.FONT_HERSHEY_SIMPLEX` specifies the font style, `1` represents the font scale (size), `(0,0,255)` is the red color in BGR format, and `2` indicates the thickness of the text strokes.

```
import cv2

cap = cv2.VideoCapture(0)

# Create fullscreen window
cv2.namedWindow("Shapes", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Shapes", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Draw rectangle
    cv2.rectangle(frame, (100,100),(300,300),(0,255,0),3)

    # Draw circle
    cv2.circle(frame,(200,200),50,(255,0,0),3)

    # Put text
    cv2.putText(frame,"OpenCV practice",(50,50),
                cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),2)

    cv2.imshow("Shapes",frame)
```

```
cv2.imshow('shapes', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
✓ 15.3s
```

Output:



8. Converting live video to find edge using canny operator:

Mathematical Equation:

Step 1: Convert Image to Grayscale

$$Gray = 0.299R + 0.587G + 0.114B$$

Gray = Intensity value of the grayscale image

R = Red channel intensity value

G = Green channel intensity value

B = Blue channel intensity value

0.299 = Weight of the red component in grayscale conversion

0.587 = Weight of the green component in grayscale conversion

0.114 = Weight of the blue component in grayscale conversion

Step 2: Noise Reduction (Gaussian Blur)

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

G(x,y) : Gaussian function value at pixel location (x,y)

x : Horizontal distance from the center of the kernel

y : Vertical distance from the center of the kernel

σ : Standard deviation controlling the amount of smoothing

π : Mathematical constant (≈ 3.14159)

e : Euler's number (≈ 2.71828)

Step 3: The image is convolved with the Gaussian kernel:

$$I_s(x, y) = I(x, y) * G(x, y)$$

$I_s(x, y)$ = Smoothed image intensity at pixel location (x,y)

$I(x, y)$ = Original image intensity at pixel location (x,y)

$G(x,y)$ = Gaussian kernel (Gaussian function)

$*$ = Convolution operation

x = Horizontal pixel coordinate

y = Vertical pixel coordinate

Step 4: Compute Image Gradient it finds G_x and G_y

The gradients in the x-direction and y-direction are computed using Sobel operators.

G_x = Gradient in the horizontal direction

G_y = Gradient in the vertical direction

Step 5: Gradient Magnitude (Edge Strength)

$$G = |G_x| + |G_y|$$

G = Gradient magnitude (overall edge strength)

G_x = Gradient in the horizontal (x) direction

G_y = Gradient in the vertical (y) direction

$|G_x|$ = Absolute value of the horizontal gradient

$|G_y|$ = Absolute value of the vertical gradient

Step 6: Gradient Direction (Edge Orientation)

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

θ = Gradient direction (edge orientation)

G_x = Gradient in the horizontal (x) direction

G_y = Gradient in the vertical (y) direction

$\tan^{-1}$ = Inverse tangent function used to compute the angle of the gradient

$G(x, y) > T_H$ - If the gradient magnitude at pixel (x, y) is greater than the high threshold T_H , the pixel is considered a strong edge.

$T_L < G(x, y) < T_H$ - If the gradient magnitude lies between the low threshold T_L and high threshold T_H , the pixel is considered a weak edge.

$G(x, y) < T_L$ - If the gradient magnitude is less than the low threshold T_L , the pixel is considered noise or non-edge and is removed.

Step 7: Non-Maximum Suppression

$$G_{NMS}(x, y) = \begin{cases} G(x, y), & \text{if } G(x, y) \geq G(x_1, y_1) \text{ and } G(x, y) \geq G(x_2, y_2) \\ 0, & \text{otherwise} \end{cases}$$

$G(x, y)$ = Gradient magnitude at pixel location (x, y)

$G(x_1, y_1)$ = Gradient magnitude of the neighboring pixel in the gradient direction

$G(x_2, y_2)$ = Gradient magnitude of the neighboring pixel in the opposite gradient direction

$G_{NMS}(x, y)$ = Gradient value after non-maximum suppression

(x, y) = Pixel coordinates in the image

Steps of Canny Edge Detection:

1. **Convert the image to grayscale** – reduce the image from RGB to a single intensity channel.
2. **Apply Gaussian blur** – smooth the image to remove noise.
3. **Calculate image gradients** – find how quickly the intensity changes in horizontal and vertical directions.

4. **Compute gradient magnitude and direction** – determine the strength and orientation of edges.
5. **Apply non-maximum suppression** – thin the edges by keeping only the strongest pixels.
6. **Apply double thresholding** – classify pixels as strong edges, weak edges, or non-edges.
7. **Perform edge tracking by hysteresis** – keep weak edges that are connected to strong edges and remove the rest.

```
import cv2

# Open webcam
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Cannot open camera")
    exit()

# Create fullscreen windows
cv2.namedWindow("Original Video", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Original Video", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

cv2.namedWindow("Canny Edge Detection", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Canny Edge Detection", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while True:
    # Capture frame
    ret, frame = cap.read()

    if not ret:
        print("Can't receive frame")
        break
```

```
if not ret:
    print("Can't receive frame")
    break

# Convert to grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Gaussian blur
blur = cv2.GaussianBlur(gray, (5,5), 0)

# Canny edge detection
edges = cv2.Canny(blur, 100, 200)

# Display frames
cv2.imshow("Original Video", frame)
cv2.imshow("Canny Edge Detection", edges)

# Press q to exit
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

✓ 11.3s

Output:



9. Converting Live video to Negative video:

```
import cv2

# Open webcam
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Cannot open camera")
    exit()

# Create fullscreen window
cv2.namedWindow("Negative Video", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Negative Video", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

while True:
    ret, frame = cap.read()

    if not ret:
        print("Can't receive frame")
        break

    # Create negative image
    negative = 255 - frame
```

```
    # Show negative video
    cv2.imshow("Negative Video", negative)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
✓ 23.6s
```

Output:



Mathematical Equation

For an 8-bit image:

$$s = 255 - r$$

Where:

- r = original pixel value
- s = negative pixel value
- 255 = maximum intensity value in an 8-bit image

10. Detecting the motion if motion found edge will be shown otherwise black screen will be shown.

Mathematical Equation:

1. Grayscale Conversion

$$Gray(x, y) = 0.299R(x, y) + 0.587G(x, y) + 0.114B(x, y)$$

$Gray(x, y)$ = Intensity value of the grayscale image at pixel location (x, y)

$R(x, y)$ = Red channel intensity value at pixel location (x, y)

$G(x, y)$ = Green channel intensity value at pixel location (x, y)

$B(x, y)$ = Blue channel intensity value at pixel location (x, y)

(x, y) = Pixel coordinates in the image

2. Frame Difference (Motion Detection):

$$D(x, y) = |G_t(x, y) - G_{t-1}(x, y)|$$

$D(x, y)$ = Difference value at pixel location (x, y)

$G_t(x, y)$ = Pixel intensity at position (x, y) in the current frame (time t)

$G_{t-1}(x, y)$ = Pixel intensity at position (x, y) in the previous frame (time $t-1$)

$| |$ = Absolute value operator

(x, y) = Pixel coordinates in the image

```

import cv2
import numpy as np

# Open webcam
cap = cv2.VideoCapture(0)

# Fullscreen window
cv2.namedWindow("Motion Edge", cv2.WND_PROP_FULLSCREEN)
cv2.setWindowProperty("Motion Edge", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

# Read first frame
ret, prev_frame = cap.read()
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convert current frame to gray
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```

# Frame difference
diff = cv2.absdiff(prev_gray, gray)

# Threshold to detect motion
_, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

motion_pixels = cv2.countNonZero(thresh)

# If motion detected
if motion_pixels > 5000:
    # Apply Canny Edge
    edges = cv2.Canny(gray, 100, 200)

    cv2.imshow("Motion Edge", edges)

else:
    # Show black screen
    black = np.zeros_like(gray)
    cv2.imshow("Motion Edge", black)

# Update previous frame
prev_gray = gray

# Quit key
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

```
cap.release()  
cv2.destroyAllWindows()
```

✓ 1m 19.0s

Output:

[Drive Output](#)

11. Number Plate Detection System

Abstract

This report details the development and implementation of a backend microservice designed to facilitate and manage remote file operations via the File Transfer Protocol (FTP). Built on the Spring Boot framework, the system exposes RESTful APIs for file uploading, image retrieval, and dynamic folder archiving, all while maintaining a persistent, database-backed audit trail. The system integrates a machine learning model and OCR engine to detect and extract vehicle number plate text from uploaded images in real time.

1. System Architecture and Technologies

The application follows a multi-layered architecture that clearly separates the web controller, business logic, and data access layers. This separation ensures maintainability, scalability, and ease of testing across all components.

- **Backend Framework:** Spring Boot (Java) provides the core infrastructure, REST API handling, dependency injection, and automatic multithreading support.
- **Database & ORM:** PostgreSQL is used as the relational database, integrated via Spring Data JPA and Hibernate. Hibernate's schema auto-generation (ddl-auto=update) initializes the database tables automatically on startup.
- **FTP Integration:** The Apache Commons Net library (FTPClient) powers low-level communication with the remote FTP server, enabling file uploads, directory navigation, and binary file writes.
- **ML & OCR:** A machine learning model detects the number plate region in the image, and an OCR engine extracts the alphanumeric characters from that region.

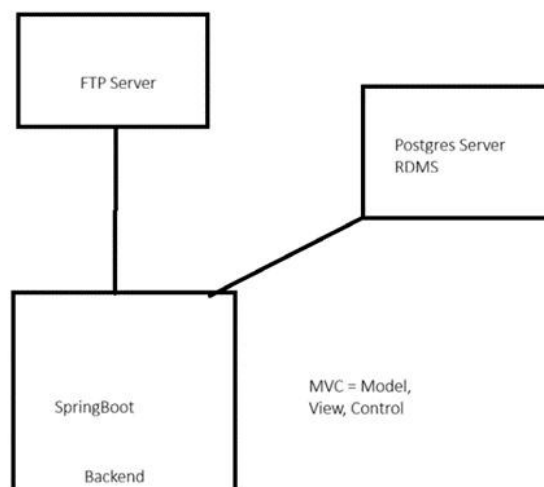
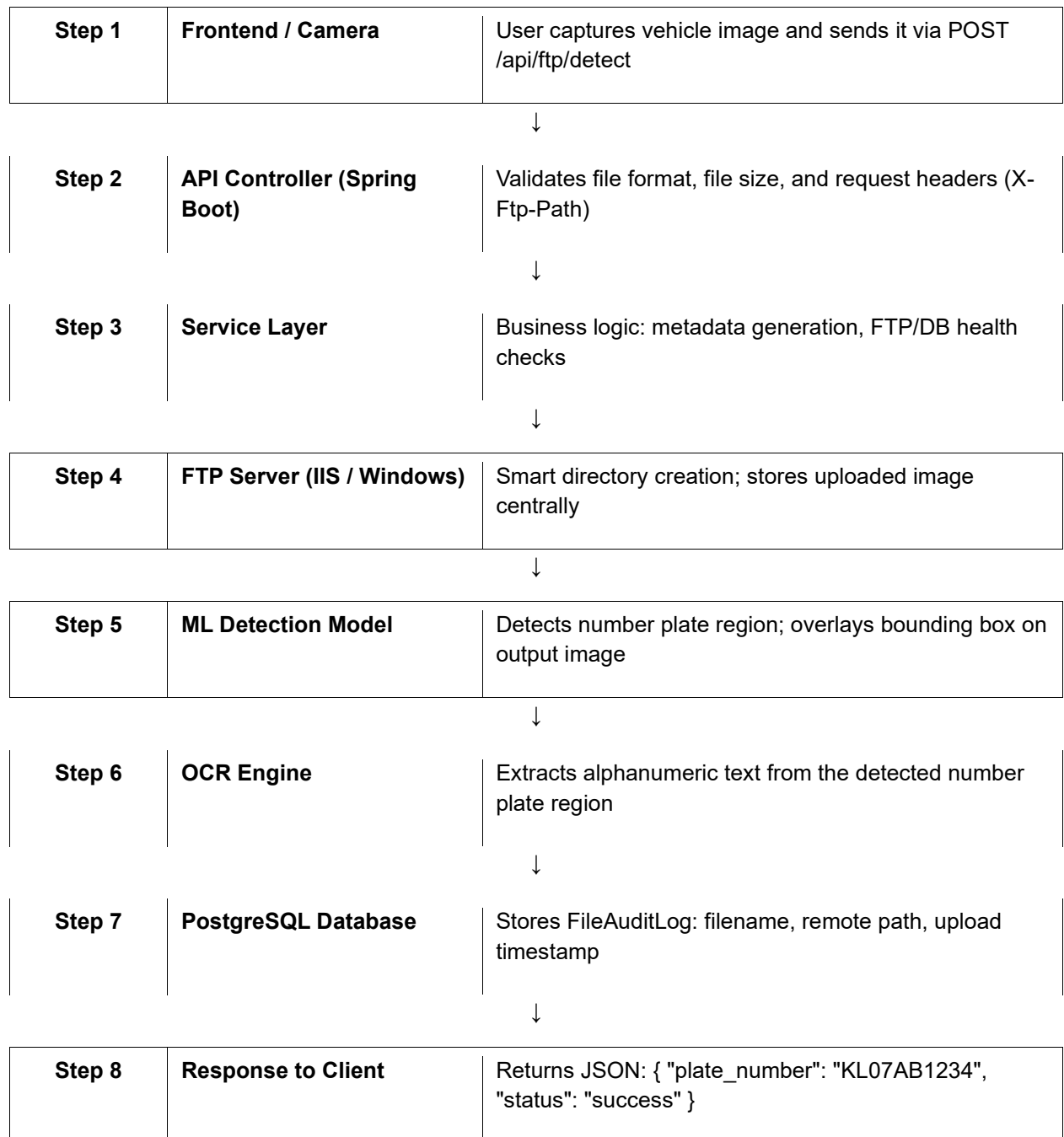


Figure 1: System Architecture — SpringBoot, FTP Server, and PostgreSQL

2. System Workflow — Flow Diagram



3. Windows Server Infrastructure Setup

To support the backend microservice, a local FTP environment was configured using Internet Information Services (IIS) on a Windows operating system. The following steps were carried out to set up the FTP server:

- The core FTP Server role and FTP Extensibility were enabled within Windows Features.

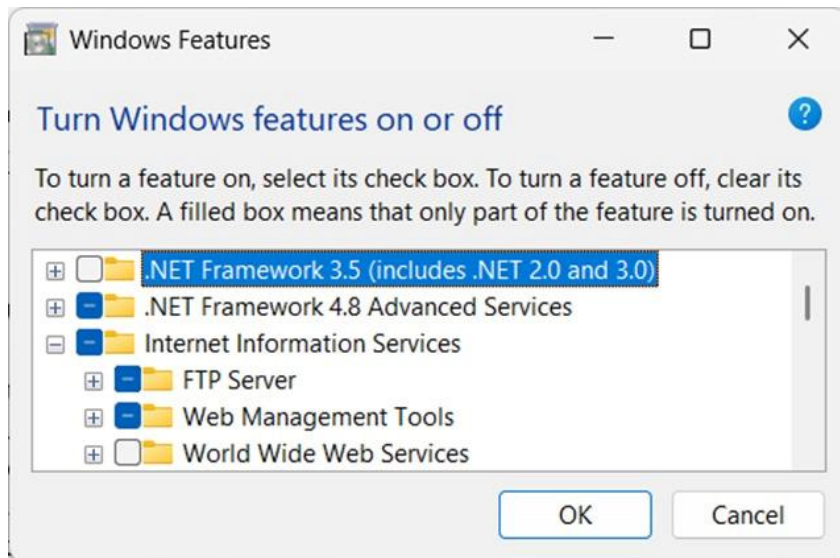


Figure 2: Enabling FTP Server role under Internet Information Services in Windows Features

- A new FTP site named EdgeFTP was provisioned in IIS Manager, bound to the standard port 21, configured to use Basic Authentication, with physical path set to D:\FTP shared.

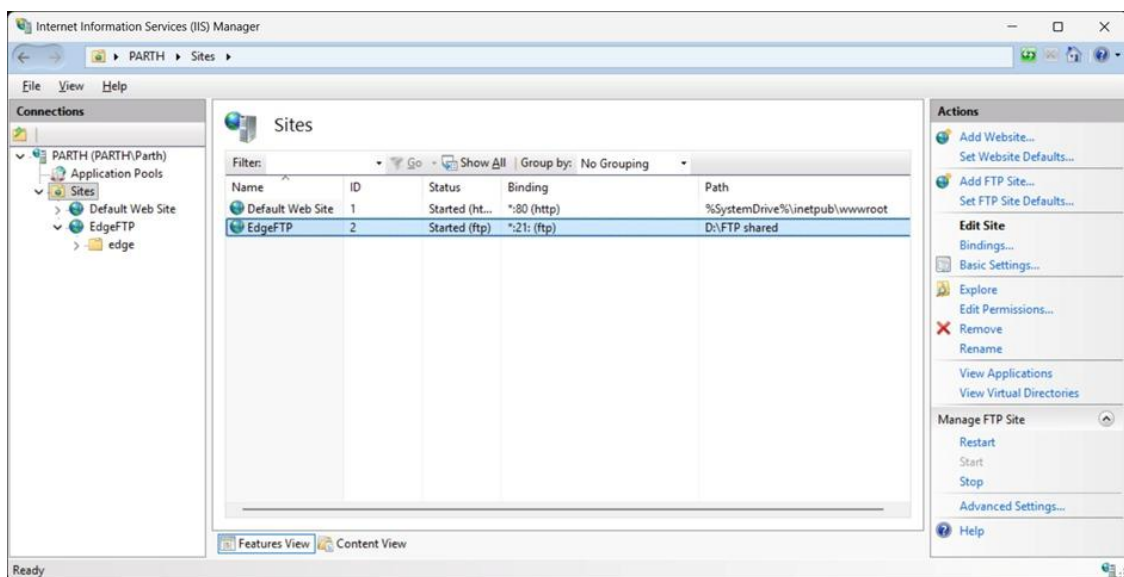


Figure 3: IIS Manager showing the EdgeFTP site running on port 21

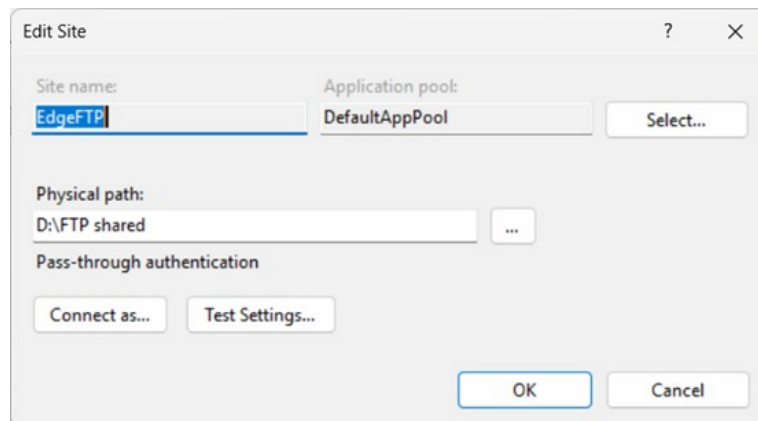


Figure 4: Edit Site dialog — EdgeFTP site configuration with physical path D:\FTP shared

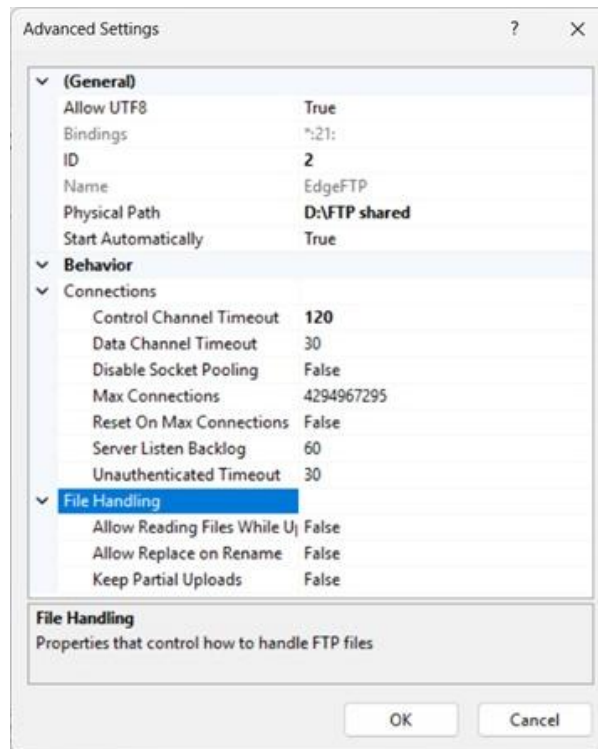


Figure 5: FTP Advanced Settings — connection timeouts and file handling properties

- Because IIS FTP relies on native Windows system accounts, a dedicated local user account was created specifically for this service via the Windows Command Prompt.
- An FTP Authorization Rule was added in IIS Manager to grant the new user explicit Read and Write permissions.
- Windows NTFS file system permissions on the physical host directory were modified to give the user Full Control, resolving all underlying authentication and access denial errors.

This configuration enabled the Spring Boot application's FTP client to successfully navigate remote directories, dynamically construct missing folders, and execute binary file writes without interruption.

4. Detailed Step-by-Step Workflow

Step 1 — Frontend Request

The process begins at the client side — a web application, mobile app, or camera system. The user captures or selects a vehicle image. This image is sent to the backend via an HTTP POST request along with a custom header specifying the target FTP path:

```
POST /api/ftp/detect
Header: X-Ftp-Path: edge/numberplate/parth
```

This API endpoint acts as the single entry point through which the frontend communicates with the backend.

Step 2 — API Controller Layer

Spring Boot's REST controller receives the incoming request and validates the following before forwarding to the service layer:

- File format is a valid image type (JPG, PNG, etc.)
- File size is within acceptable limits
- The X-Ftp-Path header is present and properly formed

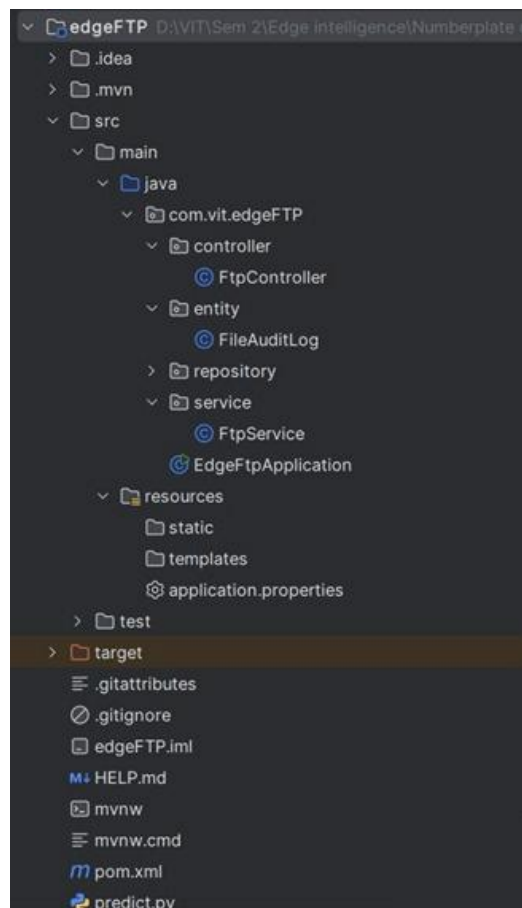


Figure 6: Spring Boot project structure — *FtpController*, *FtpService*, *FileAuditLog* entity

Step 3 — Service Layer (Business Logic)

The Service Layer handles all core operations. It first verifies that the FTP server and database connections are active, then performs the following:

- Generates metadata — unique filename and upload timestamp
- Resets the FTP working directory to the server root to avoid directory traversal errors
- Builds any missing folder structures on the FTP server recursively
- Uploads the image file to the correct remote path
- Triggers the ML detection model for number plate processing
- Saves the audit log to the PostgreSQL database

Step 4 — FTP Server (Smart Directory Management)

The uploaded image is stored on the centralized IIS FTP server. A key feature of the service layer is its smart directory management — before saving the file, it automatically checks whether the required folder structure exists on the FTP server. If any folder in the path is missing, it creates them recursively. This prevents upload failures caused by missing directories.

The FTP working directory is always reset to the root before path construction, which resolved a critical bug where nested directory navigation caused the client to lose its relative position.

Step 5 — ML Detection Model

After the image is stored, the detection model is triggered. It processes the image through the following pipeline:

- Loads the image from the FTP server
- Scans the image to identify the vehicle number plate region
- Draws a bounding box around the detected plate
- Generates an output image overlaid with the predicted bounding box and data labels
- Passes the cropped plate region to the OCR engine

Step 6 — OCR Engine (Text Extraction)

The OCR engine processes the detected number plate region and converts the characters into machine-readable text. Example output:

```
Detected Plate Text:  KL07AB1234
```

Step 7 — Database Auditing (PostgreSQL)

Every successful file upload triggers the creation of a FileAuditLog entry in the PostgreSQL database. Hibernate (via Spring Data JPA) acts as the ORM layer, allowing the application to interact with the database using standard Java objects instead of raw SQL queries. The following data is stored for every upload:

Field	Description
file_name	Original name of the uploaded image file
remote_path	The path on the FTP server where the file was saved
upload_time	Precise timestamp of when the file was successfully uploaded

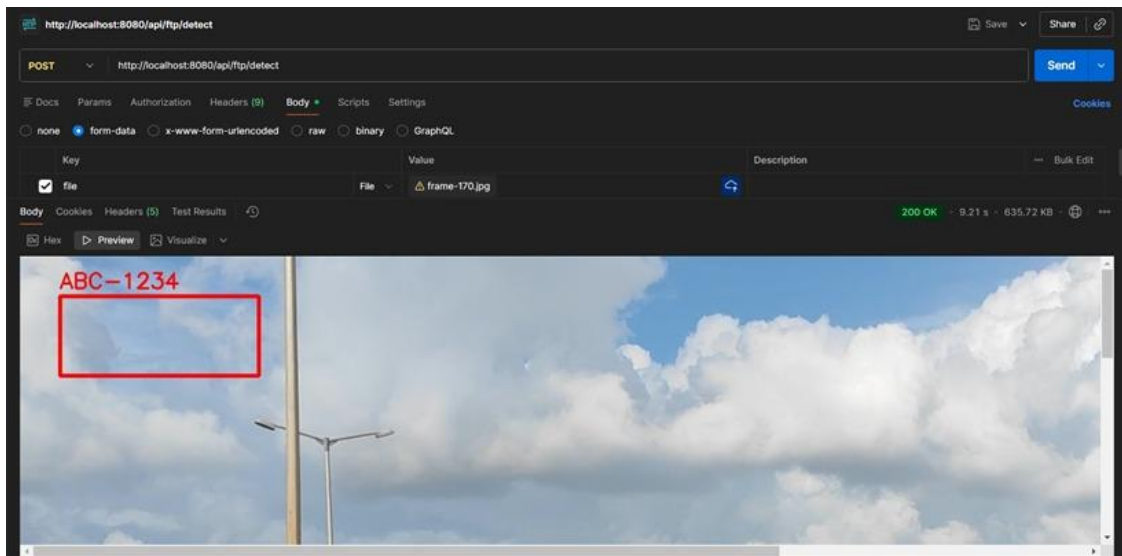


Figure 9: POST `/api/ftp/detect` — detection result showing bounding box with label ABC-1234

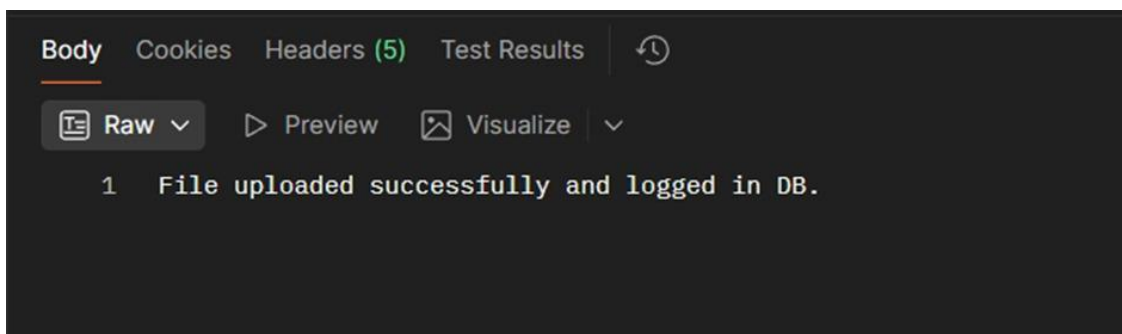


Figure 10: API success response confirming file upload and database log entry

5.2 Direct Image Retrieval — GET `/api/ftp/download/image`

This endpoint is designed for frontend asset loading. It uses the `X-File-Path` header to locate a specific file on the FTP server. The service streams the binary image data directly to the client, dynamically applying the correct HTTP Content-Type (`image/jpeg` or `image/png`) so the image renders inline in the browser.

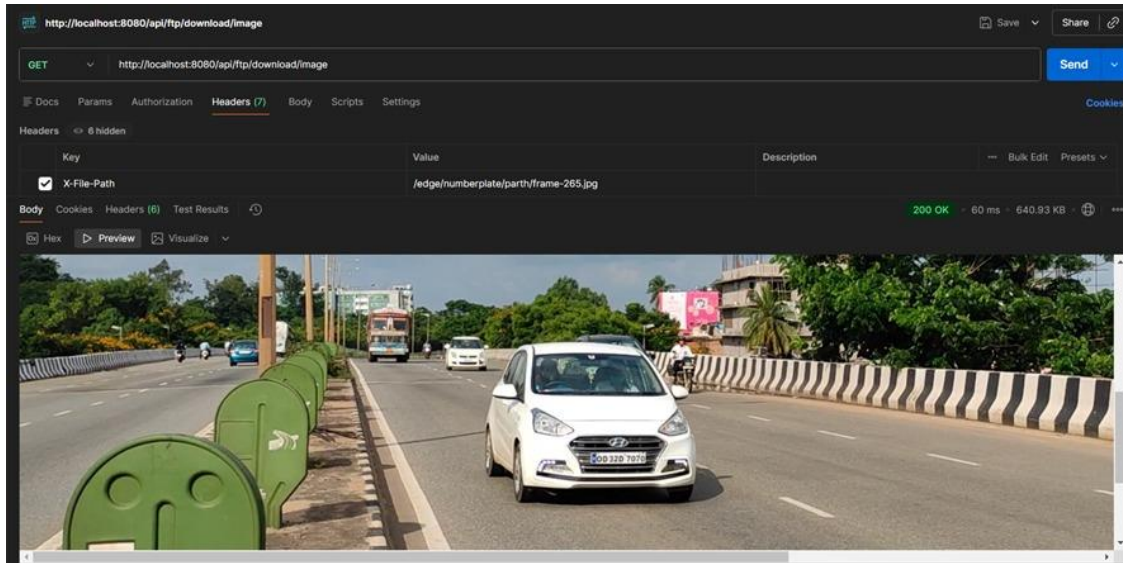


Figure 11: GET /api/ftp/download/image — retrieving and streaming an image from the FTP server

5.3 On-the-Fly Folder Archiving — GET /api/ftp/download/zip

This endpoint enables bulk data retrieval by packaging entire remote FTP directories into a single compressed archive. It iterates through all files in the target folder, streams them into a ZipOutputStream, and delivers a consolidated .zip file to the client — without requiring any intermediate local disk storage on the backend server.

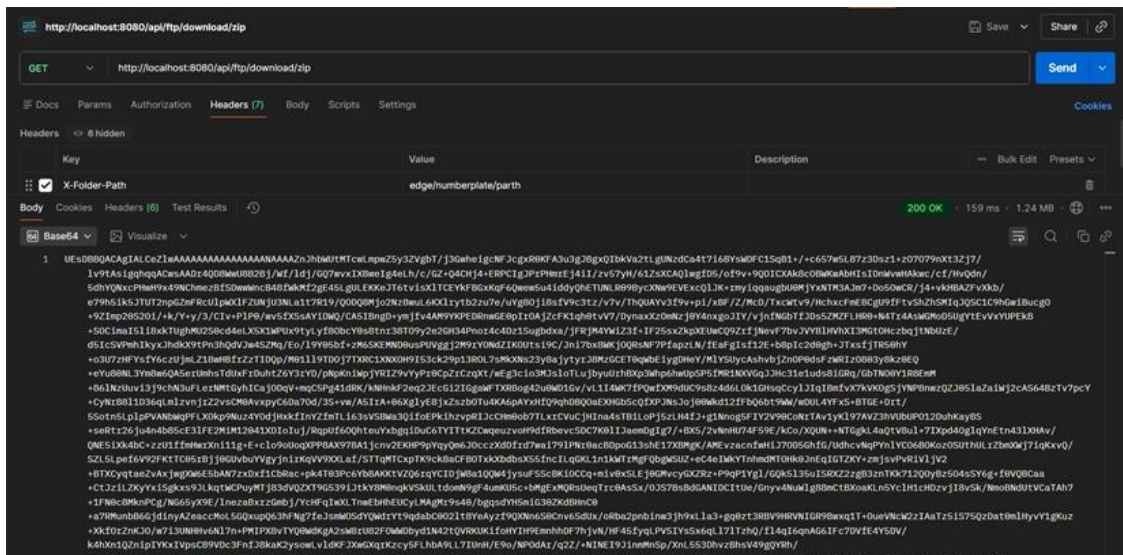


Figure 12: GET /api/ftp/download/zip — streaming a zipped archive of the FTP folder to the client

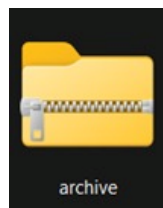


Figure 13: Downloaded .zip archive file

6. Technical Challenges and Resolutions

Challenge 1 — FTP Directory Traversal

A significant issue arose during development where standard FTP navigation logic caused the client to lose its relative position when uploading files into newly created, deeply nested directories. For example, after creating a folder two levels deep, subsequent FTP commands would fail because the working directory pointer was out of sync.

Resolution: A strict state-management protocol was implemented in the service layer. Before constructing any directory path, the working directory is explicitly reset to the FTP server root. The required folder structure is then built step by step from the root, ensuring consistent and reliable navigation at all times.

Challenge 2 — Windows IIS FTP Authentication

The IIS FTP server uses native Windows system accounts for authentication rather than its own internal user management. This caused access denial errors when the Spring Boot FTP client attempted to read and write files.

Resolution: A dedicated Windows local user account was created for the FTP service. An FTP Authorization Rule was configured in IIS Manager to grant this user explicit Read and Write permissions. Additionally, NTFS file system permissions on the physical storage directory were updated to give the user Full Control. These steps together resolved all authentication errors.

7. Deployment

The entire backend application is packaged as an executable JAR file, making it straightforward to deploy across different environments. To start the application, run:

```
java -jar number-plate-detection.jar
```

On startup, the following sequence occurs automatically:

- Spring Boot server initializes and loads all configurations
- Database connection to PostgreSQL is established; Hibernate auto-generates any missing tables
- FTP server connection is configured and verified
- All REST API endpoints become active and ready to receive requests

```

Run Main
2026-03-06T19:39:40.344+05:30 INFO 14124 --- [edgeFTP] [main] com.vit.edgeFTP.EdgeFtpApplication : Starting EdgeFtpApplication using Java 17.0.1
2026-03-06T19:39:40.357+05:30 INFO 14124 --- [edgeFTP] [main] com.vit.edgeFTP.EdgeFtpApplication : No active profile set, falling back to 1 default profile: 'default'
2026-03-06T19:39:42.365+05:30 INFO 14124 --- [edgeFTP] [main] j.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories
2026-03-06T19:39:42.488+05:30 INFO 14124 --- [edgeFTP] [main] j.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning
2026-03-06T19:39:43.835+05:30 INFO 14124 --- [edgeFTP] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-03-06T19:39:43.866+05:30 INFO 14124 --- [edgeFTP] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-03-06T19:39:43.866+05:30 INFO 14124 --- [edgeFTP] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.1]
2026-03-06T19:39:44.023+05:30 INFO 14124 --- [edgeFTP] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed
2026-03-06T19:39:44.415+05:30 INFO 14124 --- [edgeFTP] [main] org.hibernate.orm.jpa : HHN008540: Processing PersistenceUnitInfo [1]
2026-03-06T19:39:44.572+05:30 INFO 14124 --- [edgeFTP] [main] org.hibernate.orm.core : HHN000001: Hibernate ORM core version 7.2.1
2026-03-06T19:39:45.704+05:30 INFO 14124 --- [edgeFTP] [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class
2026-03-06T19:39:45.775+05:30 INFO 14124 --- [edgeFTP] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-03-06T19:39:46.184+05:30 INFO 14124 --- [edgeFTP] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql
2026-03-06T19:39:46.184+05:30 INFO 14124 --- [edgeFTP] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-03-06T19:39:46.523+05:30 INFO 14124 --- [edgeFTP] [main] org.hibernate.orm.connections.pooling : HHN10001005: Database info:
Database JDBC URL [jdbc:postgresql://localhost:5432/edge_db]
Database driver: PostgreSQL JDBC Driver

```

Figure 14: Spring Boot startup log showing successful server initialization and database connection

8. Key Technologies Used

Technology	Role in the System
Spring Boot (Java)	Backend framework; MVC architecture; REST API; dependency injection
Apache Commons Net (FTPClient)	Low-level FTP communication library for file uploads and directory management
Windows IIS FTP Server	Centralized FTP server running on Windows with Basic Authentication on port 21
ML Detection Model	Locates number plate region and overlays bounding box on output image
OCR Engine	Reads and converts plate characters to digital text
PostgreSQL	Relational database for storing audit logs and detection results
Hibernate ORM / Spring Data JPA	Object-relational mapper; auto-generates schema; removes need for raw SQL

JAR Package

Portable deployment format for the Spring Boot application

Conclusion

The finalized application serves as a highly scalable microservice that successfully bridges modern REST APIs with an FTP-based file storage system. By combining automated smart directory management, machine learning-based number plate detection, OCR text extraction, and persistent database auditing, the system provides a secure and reliable foundation for real-time vehicle number plate detection. The integration of Spring Boot, Apache Commons Net, Windows IIS FTP, PostgreSQL, and Hibernate ensures that the system operates efficiently and can scale to support multiple concurrent users without degradation in performance.